CodeInspector: Automated LLM-Supported CS1 Level Code Assessment

***Abstract*—This research-to-practice work-in-progress paper presents an automated code assessment system called *CodeInspector* for CS 1-level Java code submissions. CodeInspector is scalable and modular, leveraging open-source tools, including Jenkins, CheckStyle, PMD, JUnit, and Large Language Models, to streamline assessment workflows. Our preliminary findings indicate that CodeInspector is promising, as it reduces the assessment workloads of the instructors while providing students with immediate and formative feedback, allowing an iterative learning process. In addition, system standardization creates a more equitable learning environment.**

***Index Terms*—automated code assessment, feedback, code quality analysis, testing, large language model, software automation, Jenkins.**

I. Introduction

Timely and formative feedback is crucial in academia [1]. It allows students to reinforce learning, increase their knowledge, and develop essential skills for more advanced topics. However, manual assessment of student code submissions is time-intensive and often delays feedback [2], [3]. Such delays negatively impact student learning and performance, particularly in subsequent assignments.

Instructors often address this issue by solely focusing on code correctness and functionality while neglecting other coding aspects, such as code quality [4], [5]. However, [6] suggests that overlooking such crucial coding aspects may encourage students to lean into writing code that functions but with poor code readability, increasing the risk of bugs and maintenance costs. Even worse, instructors may address it by employing multiple teaching assistants in high-enrollment courses, often leading to inconsistent grading and feedback [3]. Therefore, these challenges require a more efficient and standardized approach to assessing programming assignments.

This study proposes ***CodeInspector***—an automated code assessment for Java programming assignments in a CS1-level course. It automates key aspects of the programming assessment process, including code quality analysis and the generation and execution of test cases, providing formative feedback on correctness, functionality, and code quality. *CodeInspector* serves as a scalable and modular solution for programming education, leveraging open-source tools and a large language model (LLM) to streamline code assessment workflows for instructors while providing immediate feedback and enabling iterative improvement for students.

The contributions of this paper include the following:

1) A scalable and modular automated assessment system that leverages open-source tools, allowing customization for various coding aspects and assessment needs.
2) The integration of a LLM for automated test case generation, enhancing the efficiency of assessment workflows.
3) An error density-based grading scheme that ensures fair assessment considering the size of the code and severity of the error.

The remainder of this paper is organized as follows. Section II presents the literature related to our research. Section III describes the proposed system design in detail. Finally, Sections IV and IV present results and analysis, conclusions, and areas of future work, respectively.

II. Related Work

A. Style Analysis

In academia, the importance of code quality cannot be overstated. Although problem solving and algorithm implementation often take precedence, the clarity and readability of the code are crucial for effective collaboration and maintenance in software development [7]. Despite this, many programming courses do not adequately emphasize code quality [4]. This is often because the time-consuming nature of manual review forces instructors to focus on assignments that assess problem-solving and algorithmic skills rather than coding standards [7], [8].

Inclusion of analysis tools in academic settings, as explored in [9], [10], provides a systematic approach for gaining insights into student code and performance. Albluwi and Salter [11] highlight the effectiveness of feedback from static analysis tools in reducing mistakes over time, indicating that these tools can play a crucial role in improving code quality education.

These academic efforts to highlight and improve code quality through teaching strategies and static analysis tools are pivotal. They enhance students' coding skills and prepare them for the collaborative and evolving nature of software development in academia and the professional world.

B. Auto-Graders

Several automated code assessment techniques and tools have been proposed for educational purposes. Cipriano et al. [12] of Lusófona University propose an automatic assessment tool used across several computer science courses. The proposed command-line tool leverages Apache Maven and Unit testing to automate software build and functionality testing.

Buffardi & Edwards [13] utilized Web-CAT to automate the evaluation of student code submissions for an introductory programming course at Virginia Tech, allowing students to repeatedly revise and re-submit their code for feedback on

their code implementation and functionality. Heckman & King [14] of North Calorina State University propose a system that utilizes Jenkins [15]—a task automation server—and GitHub—a version control system—for code assessment.

Several assessment methods were also proposed for code assessment in acadimia. [16]–[19] utilize unit testing to evaluate student code for correctness and functionality. Insa and Silva [1] provide black-box and white-box testing APIs for code assessment, along with a feature that allows students to compare the output of their code with the expected output. Peveler et al. [19] use several assessment techniques, including Linux diff tool for output comparison, checks for memory leaks, and static analysis.

C. Large Language Models (LLMs) in Test Case Generation

Integrating LLMs in test case generation represents a transformative shift in programming education, enhancing both efficiency and coverage in software testing. Traditionally, instructors manually crafted test cases, a process that was labor-intensive and prone to errors. LLMs automate this task by generating test cases directly from software requirements or user stories, significantly improving turnaround times and reducing human oversight issues [20].

Likewise, LLMs foster an interactive learning environment by delivering real-time feedback, which accelerates students' iterative coding process and reinforces software testing principles [21], [22]. Their scalability makes them particularly valuable in large programming courses, ensuring consistent and comprehensive test coverage across diverse projects. Furthermore, custom applications of LLMs support various assessment formats, facilitating innovative pedagogical approaches such as adaptive testing [23].

By catching errors early and improving software reliability, LLM-driven test case generation not only enhances student learning but also aligns with best practices in modern software development.

III. System Design

Figure 1 shows the architecture design of the proposed system. *CodeInspector* integrates Jenkins as its central component, along with a large language model for the generation of test cases. The workflow pipeline of the main components shown in the figure is discussed below.

A. Data Collection

CodeInspector offers a web-based front-end interface that allows students to submit their code for evaluation. Students can either upload their code as a Java file or write it directly using an integrated code editor. Upon submission, the code is automatically committed to the instructor's GitHub repository, triggering *CodeInspector* events.

B. Test Case Generation

The system uses a LLM to generate test cases for evaluating the correctness of the submitted code against the assignment requirements. We implemented an API using the Flask[1] framework, enabling communication between *CodeInspector* and the LLM. The open-source tool Ollama[2] is utilized to execute the LLM locally, eliminating reliance on cloud-based processing. Large Language Model Meta AI (LLaMA) [24] is a family of state-of-the-art open-weight LLMs that balance computational efficiency with advanced natural language processing (NLP) capabilities, offering researchers and developers access to powerful AI tools without extensive computational resources. Specifically, we employ Meta's llama3.2[3].

The API accepts a Java file containing an instructor's solution to a programming assignment. The LLM uses the file along with a carefully designed prompt specifying rules for generating test cases using the JUnit framework. The LLM produces a Java file containing test cases for expected inputs and edge cases, which is returned to *CodeInspector* as the API response.

C. Code Evaluation

When students submit their code, GitHub triggers a Jenkins job to perform code assessment. To streamline the build and evaluation processes, Jenkins proceeds as follows. First, it downloads dependencies to set up the execution environment. Next, it retrieves the code submitted from the GitHub repository along with an *Ant*[4] build script and the generated test file. Finally, it executes the Ant script to compile the code, perform unit testing and a style analysis on the source code.

We selected the JUnit[5] library for black-box testing and used CheckStyle[6] and PMD[7] for code style analysis. Both CheckStyle and PMD are highly scalable and configurable, enabling instructors to select the checks should only be considered when analyzing students' code. To ensure relevance, we configure these tools to check for a small set of coding standards suitable for novice programmers. Due to space limits, Table I provides a very brief description of the coding standards from the grading rubric used for the Java code submissions; for a detailed description, see [10], [25].

D. Feedback Generation

Once the code evaluation process is complete, *CodeInspector* utilizes the reports from Junit and the style analysis tools, highlighting detected style violations and implementation defects, to create more formative feedback supporting an iterative learning process. The system then emails the student the report, allowing them to revise their code and resubmit it for additional feedback.

E. Auto-grading

In this stage, the student code is graded according to the criteria described below. Each criterion is assessed using a tiered system, ranging from Excellent to Unsatisfactory. The

[1] https://flask.palletsprojects.com/en/stable/

[2] https://ollama.com

[3] https://www.llama.com/llama-downloads/

[4] https://ant.apache.org

[5] https://junit.org/junit5/

[6] https://checkstyle.sourceforge.io

[7] https://pmd.github.io

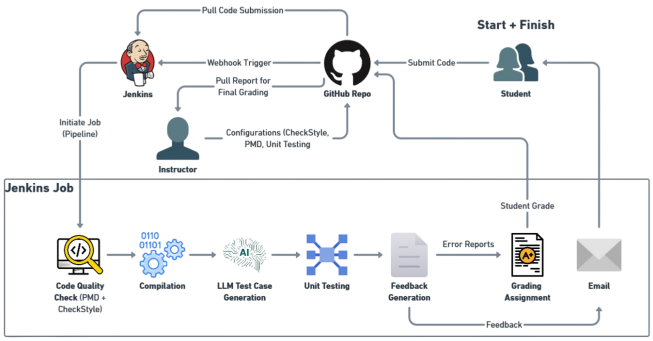

Fig. 1. *CodeInspector* design.

TABLE I
CODING STANDARDS GRADING CRITERION

Criterion	***Description***
Javadoc Documentation	The code is well documented, including proper JavaDoc tags that comment on the author's name, date, program's task, and JavaDoc methods that effectively document the program's methods.
Comments	Comments specifying the objective of each method and control flow constructs are present and meaningful.
Naming Conventions	The names of, the program, methods, and variables are descriptive and adheres to coding standards
Whitespace	There is proper indentation and white space; Code lines are not excessively long (less than 100.)

assigned tier is determined by a percentage threshold and corresponds to a specific portion of the total grade of the criterion (see [10]).

1) Requirements: This criterion evaluates how thoroughly the submitted code meets the assignment's functional requirements. The grade calculation for this criterion is straightforward. Jenkins runs JUnit tests and then calculates the pass rate to determine which points should be assigned from the tiered rubric.

2) Coding Standards: Unlike the requirement criterion, the style error distributions can vary significantly from submission to submission. Therefore, our grading approach for this criterion ensures the following:

- Since errors have different severity levels, ranging from high, medium, and low severity, the grading process should ensure fairness using a penalty-based approach instead of threshold scoring. Therefore, each error is penalized according to its severity classification.
- The correlation between code size and error rate should be considered when assigning penalties. Smaller or incomplete submissions may result in fewer violations, while other submissions may result in a higher violation rate due to their code size. Therefore, assigning penalties solely based on total violations without considering the code size is unfair and may discourage novice programmers from writing large code to avoid higher penalties. Our grading scheme addresses this issue by basing penalties on error density rather than total violations.

Algorithm 1: Error Density, Adjustment-based Grading Algorithm

Input: Max_Grade, Min_Grade, Max_Deduction, Penalty_per_Category, Errors_per_Category, LOC
Output: Final_Grade
Let Total_Penalty ← 0;
foreach *category* ∈ *Errors_per_Category[category]* **do**
  Absolute_Penalty ← Penalty_per_Category[category] × Errors_per_Category[category];
  Error_Density ← Errors_per_Category[category] ÷ LOC;
  Adjusted_Penalty ← Absolute_Penalty × Error_Density;
  Total_Penalty ← Total_Penalty + Adjusted_Penalty;
end
Final_Grade ← max(Max_Grade - min(Adjusted_Penalty, Max_Deduction), Min_Grade);
return *Final_Grade*;

Algorithm 1 implements the scheme that *CodeInspector* uses to determine the grade of the coding quality of the submitted code. The grading process begins by computing the absolute penalty for each error category, which is determined by multiplying the total style violations within the error category by the corresponding penalty weight. The algorithm then computes the error density to normalize the penalty relative to the size of the code. The error density is the number of errors per line of code (LOC). The adjusted penalty is then obtained by scaling the absolute penalty using the error density, ensuring that students with larger code sizes are not unfairly penalized for having more violations.

Following this, the adjusted penalty is subtracted from the Max Grade, and the difference is capped at the Max Deduction to prevent excessive penalties. The result is then compared to the Min Grade, and the final grade is the higher value between the two, ensuring that the student's Grade cannot fall below the predefined minimum threshold.

3) Runtime: Similarly to the requirements criterion, Jenkins runs unit and edge case testing to assess the code's correctness and behavior. The grade assigned from the grading rubric must correspond to the test pass rate.

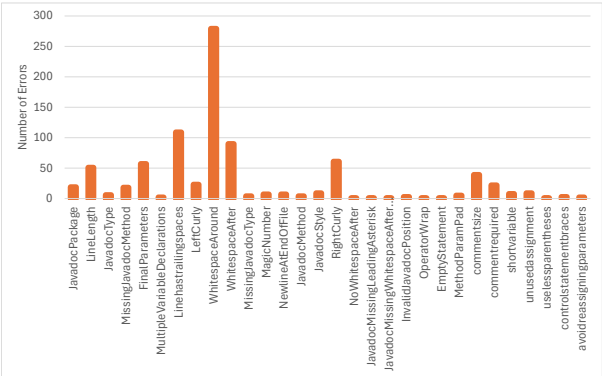

Fig. 2. Frequency of detected style errors.

```
=======================   CODING STANDARDS
==================================================

CHECKSTYLE ---------------------------------------------

An overall of 121 violations were detected by CheckStyle.

Breakdown by severity level:

 Severity: info - 15 violations
  Most frequent category: regexp (11 occurrences)
  Most frequent type: RegexpSingleline (11 occurrences)
  Example from file: CardDescription.java, in line
      []  Feedback: Line has trailing spaces.
 Severity: warning - 98 violations
  Most frequent category: whitespace (98 occurrences)
  Most frequent type: WhitespaceAroundCheck (77 occurrences)
  Example from file: CardDescription.java, in line
      []  Feedback: 'if' is not followed by whitespace.
 Severity: error - 8 violations
  Most frequent category: checks (3 occurrences)
  Most frequent type: FinalParametersCheck (3 occurrences)
  Example from file: CardDescription.java, in line
      []  Feedback: Parameter rank should be final.

Overall weighted CheckStyle error density: 0.02758
This places you in the 83rd percentile of submissions in the database.
That is, 17% of submissions had a higher CheckStyle error density than yours.
```

Fig. 3. CheckStyle results from the generated feedback.

IV. Results and Analysis

In this section, we provide a detailed analysis of our evaluation process to assess the effectiveness of the proposed system. Students were assigned a benchmark task to implement a method that converts the rank and suit of a playing card from shorthand notation to its full descriptive form. A total of 19 submissions, comprising 1,315 lines of code, were evaluated using CodeInspector for code quality and correctness, with a copy of each assessment result automatically emailed to the corresponding student.

Figure 2 shows the frequency of coding standards violations identified by CheckStyle and PMD within CodeInspector, totaling 843 errors. As shown, WhiteSpaceAround, LineHasTrailingSpaces, and WhiteSpaceAfter are most frequently presented in all submissions. While these whitespace errors are harmless to code functionality, they impact code readability and collaboration among team members, making it challenging for reviewers to comprehend modifications during code reviews and maintenance.

Figure 3 presents a report of code style violations generated by *CodeInspector* for a student code submission, showcasing the system's capacity to deliver granular, actionable feedback to students. Of the 121 errors detected by CheckStyle, 15 were classified as "info", 98 as "warning", and 8 as "error", emphasizing critical issues that require immediate attention. Notably, the most frequent violations across these categories were whitespace related errors—particularly WhiteSpaceAround and LineHasTrailingSpaces (88 occurrences)—as well as FinalParametersCheck (3 occurrences), which highlights a recurring pattern of non-final parameters in methods, a common pitfall in early-stage programming education. The report's structured breakdown by severity and category enables instructors to prioritize feedback while empowering students to iteratively refine their code based on clear, rule-based guidelines.

CodeInspector's integration of CheckStyle further demonstrates its utility in contextualizing performance: the reported weighted error density (0.02758) situates the submission in the 83rd percentile of analyzed submissions, indicating that the submission had a higher rate of code style violations than 83% of its peers, reflecting weaker adherence to coding standards. This comparative metric underscores the system's ability to provide not only formative feedback but also benchmarking insights, fostering self-awareness and motivating continuous improvement. Such automated, data-driven assessments alleviate instructors' grading burdens while ensuring consistent, objective evaluation—a critical step toward scalable, high-quality coding education.

Figure 4 illustrates the efficacy of *CodeInspector*'s automated testing framework, powered by JUnit and test cases generated by the LLM, in evaluating functional correctness and assigning holistic scores to student submissions. In the report we can see that of the 11 unit tests executed, 6 cases passed successfully, while 5 failed for various reasons highlighted in the feedback, including a discrepancy between the expected string format ([Five] of Diamonds) and the actual output ([5] of Diamonds). This granular feedback exemplifies how *CodeInspector* leverages LLM-driven test generation to pinpoint logical errors, enabling students to iteratively refine their code against precise, real-world expectations.

```
============================   REQUIREMENTS + RUNTIME
=====================================

JUNIT ---------------------------------------------

An overall of 5 out of 11 unit tests failed.

The following are the ones that failed:
    Test: testGetCardDescription_FiveOfDiamonds failed with the error:
expected:<[Five] of Diamonds> but was:<[5] of Diamonds>
    Test: testGetCardDescription_InvalidRank failed with the error:
expected:<[Invalid card rank]> but was:<[ of Diamonds]>
    Test: testGetCardDescription_InvalidSuit failed with the error:
expected:<[Invalid card suit]> but was:<[Ace of ]>
    Test: testGetCardDescription_EmptyRank failed with the error:
expected:<[Invalid card rank]> but was:<[ of Diamonds]>
    Test: testGetCardDescription_EmptySuit failed with the error:
expected:<[Invalid card suit]> but was:<[Ace of ]>

================================  FINAL SCORE
=========================================
    - Requirements: 36
    - Coding Standards: 9
    - Runtime: 12
==========================================================================
===================
    - Overall: 57
========================================= END REPORT
```

Fig. 4. Unit test and final score results from the generated feedback.

Moreover, the final score breakdown further underscores *CodeInspector*'s modular evaluation design, balancing multi-

ple assessment criteria (e.g., requirements fulfillment, coding standards, runtime performance) to reflect holistic competency. For this particular submission, *CodeInspector* yields an overall score of 57, demonstrating its ability to synthesize diverse evaluative dimensions into actionable insights. This structured, automated scoring mechanism not only streamlines grading but also ensures transparency and consistency, aligning with pedagogical goals of fostering both technical rigor and iterative improvement in student work.

Conclusion

CodeInspector represents a transformative advancement in automated code assessment, seamlessly integrating open-source tools (Jenkins, CheckStyle, PMD, JUnit) and Large Language Models to deliver scalable, modular, and pedagogically aligned evaluations. By leveraging GitHub-triggered Jenkins pipelines and Apache Ant scripts, the system automates the entire build and evaluation lifecycle—from environment setup to code compilation, unit testing, and style analysis—ensuring consistency and efficiency at scale. The strategic selection of configurable tools like CheckStyle and PMD, tailored to novice programmers' needs, underscores CodeInspector's commitment to fostering foundational coding practices while minimizing cognitive overload.

Our preliminary findings demonstrate that CodeInspector not only reduces instructors' grading burdens but also empowers students through granular, actionable feedback. The holistic scoring framework, combining requirements fulfillment, coding standards, and runtime performance, reflects a nuanced understanding of competency, even as efficiency metrics remain customizable for diverse curricula.

CodeInspector paves the way for adaptive, data-driven education. Its modular design enables seamless updates to accommodate evolving standards or emerging technologies, ensuring long-term relevance. Future work may explore deeper integration of AI-driven feedback personalization and expanded efficiency. Likewise, we want to study the effect of using CondeInspector withing several groups of students to gather their feedback while using the system.

References

[1] David Insa and Josep Silva. Automatic assessment of java code. *Computer Languages, Systems Structures*, 53:59–72, 2018.

[2] Andre LC Barczak, Anuradha Mathrani, Binglan Han, and Napoleon H Reyes. Automated assessment system for programming courses: a case study for teaching data structures and algorithms. *Educational technology research and development*, 71(6):2365–2388, 2023.

[3] Ramon Lawrence, Sarah Foss, and Tatiana Urazova. Evaluation of submission limits and regression penalties to improve student behavior with automatic assessment systems. *ACM Trans. Comput. Educ.*, 23(3), June 2023.

[4] Hsi-Min Chen, Wei-Han Chen, and Chi-Chen Lee. An automated assessment system for analysis of coding convention violations in java programming assignments. *J. Inf. Sci. Eng.*, 34:1203–1221, 2018.

[5] Markus Hofbauer, Christoph Bachhuber, Christopher Kuhn, and Eckehard Steinbach. Teaching software engineering as programming over time. In *2022 IEEE/ACM 4th International Workshop on Software Engineering Education for the Next Generation (SEENG)*, pages 51–58, 2022.

[6] Xiaosong Li and Christine Prasad. Effectively teaching coding standards in programming. In *Proceedings of the 6th Conference on Information Technology Education*, SIGITE '05, page 239–244, New York, NY, USA, 2005. Association for Computing Machinery.

[7] Oscar Karnalim and Simon. Promoting code quality via automated feedback on student submissions. In *2021 IEEE Frontiers in Education Conference (FIE)*, pages 1–5, 2021.

[8] Oscar Karnalim, Simon, and William Chivers. Work-in-progress: Code quality issues of computing undergraduates. In *2022 IEEE Global Engineering Education Conference (EDUCON)*, pages 1734–1736, 2022.

[9] Stephen H. Edwards, Nischel Kandru, and Mukund B.M. Rajagopal. Investigating static analysis errors in student java programs. In *Proceedings of the 2017 ACM Conference on International Computing Education Research*, ICER '17, page 65–73, New York, NY, USA, 2017. Association for Computing Machinery.

[10] Essa Imhmed, Edgar Ceh-Varela, and Scott Kilgore. Identifying code quality issues for undergraduate students using static analysis and nlp. In *2023 International Conference on Computational Science and Computational Intelligence (CSCI)*, pages 1527–1533, 2023.

[11] Ibrahim Albluwi and Joseph Salter. Using static analysis tools for analyzing student behavior in an introductory programming course. *Jordanian Journal of Computers and Information Technology (JJCIT)*, 6(3):215–233, 2020.

[12] Bruno Pereira Cipriano, Nuno Fachada, and Pedro Alves. Drop project: An automatic assessment tool for programming assignments. *SoftwareX*, 18:101079, 2022.

[13] Kevin Buffardi and Stephen H. Edwards. Effective and ineffective software testing behaviors by novice programmers. ICER '13, page 83–90, New York, NY, USA, 2013. Association for Computing Machinery.

[14] Sarah Heckman and Jason King. Developing software engineering skills using real tools for automated grading. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*, SIGCSE '18, page 794–799, New York, NY, USA, 2018. Association for Computing Machinery.

[15] J. Hembrink and P-G. Stenberg. Continuous integration with jenkins coaching of programming teams (eda 270). 2013.

[16] Mario Amelung, Katrin Krieger, and Dietmar Rösner. E-assessment as a service. *IEEE Transactions on Learning Technologies*, 4(2):162–174, 2011.

[17] Sebastian Pape, Julian Flake, Andreas Beckmann, and Jan Jürjens. Stage - software tool for automatic grading of testing exercises - case study paper. In *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, pages 491–500, 2016.

[18] Brad Vander Zanden and Michael W Berry. Improving automatic code assessment. *Journal of Computing Sciences in Colleges*, 29(2):162–168, 2013.

[19] Matthew Peveler, Jeramey Tyler, Samuel Breese, Barbara Cutler, and Ana Milanova. Submitty: An open source, highly-configurable platform for grading of programming assignments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education*, pages 641–641, 2017.

[20] Nischal Ashok Kumar and Andrew Lan. Using large language models for student-code guided test case generation in computer science education. *arXiv preprint arXiv:2402.07081*, 2024.

[21] Weimin Xiong, Yiwen Guo, and Hao Chen. The program testing ability of large language models for code. *arXiv preprint arXiv:2310.05727*, 2023.

[22] Felix Dobslaw, Robert Feldt, Juyeon Yoon, and Shin Yoo. Challenges in testing large language model based software: A faceted taxonomy. *arXiv preprint arXiv:2503.00481*, 2025.

[23] Zhiyi Xue, Liangguo Li, Senyue Tian, Xiaohong Chen, Pingping Li, Liangyu Chen, Tingting Jiang, and Min Zhang. Llm4fin: Fully automating llm-powered test case generation for fintech software acceptance testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1643–1655, 2024.

[24] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[25] Essa Imhmed, Edgar Ceh-Varela, Hashim Abu-Gellban, and Scott Kilgore. Fostering code quality practices among undergraduate novice programmers. *J. Comput. Sci. Coll.*, 39(7):21–32, April 2024.